

Chapter 7: Architecture Repository

Overview

Chapter 7 (pp. 127–187) of **Mastering the TOGAF® Standard** by Eric Jager explores the Architecture Repository — described by the TOGAF Standard as an **intangible concept**, not a tool, in which all architecture assets, reusable building blocks, an organization's metamodel, and governance information are collected and organized. The chapter walks through the repository's structural partitions (Architecture Metamodel, Architecture Landscape, Solutions Landscape, Architecture Requirements Repository, Reference Library, Standards Library, Governance Repository, Architecture Capability), the three lenses used to view its contents (the Enterprise, Architecture, and Solutions Continua), the building blocks that populate it (Architecture Building Blocks and Solution Building Blocks), the documentation techniques used to capture content (catalogs, diagrams, maps, matrices), the concepts of views and viewpoints used to communicate architecture, and finally practical guidance on choosing a tool to make the repository tangible. This is part of the book's GREEN reading tour (per Figure 1-1 in Chapter 1, Section 1.3).

Detailed Section Breakdown

7.1 The Central Hub (pp. 128–129)

Many organizations conflate architecture **tools** (spreadsheets, slide decks, or robust applications) with the Architecture Repository itself, because these tools often store diagrams and visualizations built up over time. However, the TOGAF Standard is explicit that the **Architecture Repository is an intangible concept** — it is not a tool and should not be directly compared to one. A well-designed repository should only contain **reusable building blocks, an organization-specific metamodel, and additional architecture governance information**; it should **not** contain diagrams of specific point solutions.

The Architecture Repository functions as the **backbone or central hub** for the Enterprise Architecture practice, bringing together frameworks, templates, and reference materials in a structured environment. It organizes both completed artifacts and works in progress, and it acts as

an *input* to the perspective provided by the three continua (Enterprise, Architecture, and Solutions). Figure 7-1 (p. 129) visualizes the repository's key components, each of which serves its own distinct purpose.

7.2 Key Components of the Architecture Repository (pp. 129–135)

This is the core structural section of the chapter, walking through Figure 7-1 from top to bottom. Each component is defined and then elaborated:

Architecture Metamodel (p. 129): Defines the structure for how architecture artifacts are organized and how different types of information (business, data, application, technology, and security) relate to each other. It ensures consistency across architecture projects, gives architects a common language, and helps align teams across the organization. The **Enterprise Metamodel** (from Chapter 5) serves as the foundation for the Architecture Metamodel and guides the creation of the Architecture Landscape.

Architecture Landscape (p. 130): A view of the organization's current, planned, and aspirational architecture, typically including different levels of architecture (strategic, segment, capability) and timelines (past, present, future). Because it can grow very large and must serve diverse stakeholders, it is divided into **three levels of granularity** (Figure 7-2, p. 131):

- **Strategic Architectures** — long-term, organization-wide summary views providing an organizing framework for operational and change activities; enable executive-level direction setting.
- **Segment Architectures** — more detailed operating models for specific areas of the organization, used at the program/portfolio level to organize detailed change activities; also called **Domain Architectures** (elaborated further in Chapter 10, Section 10.3.2, Figure 10-7).
- **Capability Architectures** — detailed views of how the organization supports a particular unit of capability; show current capability, target capability, and capability increments, enabling work packages/projects to be grouped into managed portfolios and programs.

These three levels also play an important role in agile architecture approaches (detailed further in Chapter 18). The Architecture Landscape provides a container for **Architecture Building Blocks**, which are created during architecture work by following the ADM.

Solutions Landscape (pp. 131–132): Contains models and documents related to individual Solution Architectures; more detailed than the Architecture Landscape, enabling the creation of specific system or business solutions using **Solution Building Blocks**. These Solution Building Blocks support the Architecture Building Blocks specified, developed, and deployed during ADM phases A, B, C, and D, and can be categorized as Strategic, Segment, or Capability Solution Building Blocks (mirroring the Architecture Building Block categories). Important nuance: the Solutions Landscape does **not** include the actual content/data produced by the solutions themselves — that is the responsibility of the solutions.

Architecture Requirements Repository (pp. 132–134): Captures, manages, and tracks architecture requirements, both business and technical, ensuring alignment with actual business needs and consistent requirements management across phases/projects. All requirements created or updated during the ADM are captured here. Requirements are stored corresponding to the three landscape levels:

- **Strategic Architecture Requirements** — forward-looking, executive-level; outline adjustments needed to support strategic direction; act as a roadmap for the organization's vision and evolution.
- **Segment Architecture Requirements** — precise, content-focused, portfolio/project-level guidance that breaks strategic requirements into actionable steps; the "tactical enablers" translating high-level goals into concrete actions.
- **Capability Architecture Requirements** — focused on frameworks tied to specific capabilities or groups of capabilities; define needs of individual initiatives/projects/work packages within a broader portfolio.

Reference Library (p. 134): Contains best practices, guidelines, templates, and patterns reusable across architecture projects — a ready-to-use resource accelerating architecture development. It includes reference architectures/models, templates, and the **Viewpoint Library** (a collection of views and viewpoints). Reusing existing solutions saves time and keeps solutions standardized. The **Architecture Continuum** (Section 7.3.2) is the mechanism used to classify these reference materials, from Foundation and Common Systems Architectures through to Industry and Organization-Specific Architectures.

Standards Library (pp. 134–135): A collection of standards and policies guiding architecture development and governance — IT standards, technology protocols, business process standards, and industry guidelines. Standards include legal/regulatory mandates as well as industry and organizational standards. Both the Reference Library and Standards Library are fed by **external reference models and standards**, e.g., the **BIZBOK Guide** (Business Architecture Guild Body of

Knowledge, an external reference model) and the **TOGAF Standard itself** (an example of an external standard). The Standards Library ensures compliance and consistency and helps avoid redundancies/conflicts between architecture components and projects.

Governance Repository (p. 135): A repository of decisions, rules, governance guidelines, and principles applied to architecture projects, including records of approvals, waivers, and compliance assessments. Together with the Standards Library, it ensures projects adhere to internal/external requirements, prevents non-compliance, and provides accountability and traceability for architectural decisions — critical for Risk Management and regulatory compliance. Note: the Architecture Landscape is governed by **both** the Governance Repository **and** the Architecture Capability.

Architecture Capability (pp. 135–136): Refers to the skills, processes, organizational structures, and roles that support the development and management of Enterprise Architecture. It helps manage resources efficiently and supports continuous improvement of the EA practice. Both the Governance Repository and the Architecture Capability gain visibility through the **Architecture Board**, which also acts as an escalation mechanism; conversely, the Architecture Board provides direction and management of the Architecture Capability.

The section closes by framing the Architecture Repository as a powerful tool for managing architectural assets and supporting governance, compliance, and collaboration — accelerating project delivery and ensuring consistency. (Further Reading: TOGAF Standard 10th Edition, document C220.)

7.3 The Three Continua (pp. 136–149)

A **continuum** is introduced as a conceptual framework/lens for viewing the Architecture Repository — it enables a modular, adaptable, scalable approach to shaping architecture, letting architects adjust detail, complexity, and governance to specific needs. The framework distinguishes **three continua** (Figure 7-3, p. 137): the **Enterprise Continuum**, **Architecture Continuum**, and **Solutions Continuum**. Applied left to right, they provide an increasingly detailed perspective of the repository's contents.

- **Enterprise Continuum (p. 138):** An umbrella term covering both the Architecture Continuum and Solutions Continuum, bridging conceptual architectures and practical solutions. It

provides structure for understanding how assets are developed and reused from the highly abstract to the practical, and supports reuse of architectural assets like building blocks.

- **Architecture Continuum (p. 138):** Guides the creation and evolution of architectures via a range of options from generalized/high-level to specific/concrete. Lets architects organize/classify architectural assets as the organization matures.
- **Solutions Continuum (p. 138):** Works in parallel with the Architecture Continuum but focuses on delivering actual, real-world solutions/products aligned with the architecture — from generic solutions to specific implementations.

Together, the three continua give a flexible approach supporting scalability, reuse, and customization. The Architecture Repository is the centralized hub for all three, storing artifacts, standards, and references and serving as the go-to resource for retrieval, version management, and governance controls (Figure 7-4, p. 139, highlights where in the repository the Architecture and Solutions Continua perspectives are concentrated).

7.3.1 The Enterprise Continuum (pp. 139–141)

The Enterprise Continuum is a conceptual framework/lens over the **entire** collection of architecture assets stored in the repository — architecture descriptions, models, building blocks, patterns, viewpoints — including assets from the broader industry, not just the organization. It shows the evolution of related architectures **from generic to specific, from abstract to concrete, and from logical to physical**. It is the **outermost** continuum, classifying contextual assets used to develop architectures (policies, standards, strategic initiatives, organizational structures, organization-level capabilities). It can also classify solutions (not just descriptions of them). It contains two specializations: the Architecture Continuum and the Solutions Continuum, and it provides a scale from most general (foundation) to most specific (organization-specific) — visualized in Figure 7-5 (p. 140), "the four classification models of the Enterprise Continuum." A sidebar (p. 141) reminds the reader that the Enterprise Metamodel from Chapter 5 is itself a **Foundation** Enterprise Metamodel — a starting point requiring organization-specific modification, not a single complete truth.

7.3.2 The Architecture Continuum (pp. 141–146)

This is the structuring/categorization mechanism for storing architecture artifacts, standards, and reference architectures in the Architecture Landscape, Reference Library, and Standards Library. It consists of **four architecture types**, ranging from foundational (generic) to organization-specific

(tailored) (Figure 7-6, p. 142). A book-author aside notes Figure 7-6 can be hard to interpret since the model shapes of Foundation/Common Systems Architectures aren't visually reflected in the Industry/Organization-Specific models — except that parts of the Industry Architectures model *do* reappear in the Organization-Specific Architectures model.

1. **Foundation Architectures (pp. 142–143):** The most generalized/abstract type; generic components, interrelationships, principles, and guidelines forming a foundation for more specific architectures. High-level reference models for universal services/technologies usable across industries (e.g., network protocols, operating systems, security frameworks, service-oriented architectures) — not specific to any industry or organization.
2. **Common Systems Architectures (p. 143):** Build on Foundation Architectures but remain generic across multiple industries; focus on more specific, common solutions applicable across domains (e.g., content management systems, CRM, data analytics platforms). Each is incomplete for overall system functionality but *complete* for a particular problem domain (security, manageability, networking, operations). Healthcare example: general patient management, scheduling, or billing/invoicing systems — usable across industries but must meet healthcare-specific regulations.
3. **Industry Architectures (pp. 143–144):** Tailored to a specific industry's unique processes, requirements, and regulations; provide reference models, guidelines, standards, and best practices/common frameworks for industry-specific capabilities. Healthcare example: EHR systems, telemedicine platforms, health information exchange (HIE) standards, HL7 data-exchange standards, ICD-11 medical coding.
4. **Organization-Specific Architectures (p. 144):** The most tailored/customized type, designed for a particular organization's unique goals, strategies, environment, workflows, processes, and culture. Healthcare example: a hospital's customized EHR system integrating a custom patient portal, local pharmacy system integration, population-health tools, mobile health app configuration, telemedicine support, and external lab integration.

Figure 7-7 (p. 145) shows the **four steps for applying** the four architecture types in sequence:

5. Start with **Foundation Architectures** to ensure components (infrastructure, security) are built on widely accepted standards.
6. Move to **Common Systems Architectures** to adopt general-purpose systems (e.g., billing, scheduling software).
7. Use **Industry Architectures** to tailor systems to industry-specific requirements (e.g., patient records, regulatory compliance for healthcare).
8. Develop **Organization-Specific Architectures** to customize systems to the specific organization's needs.

A personal sidebar (p. 146) recounts the author's experience implementing EA for hospitals in the Netherlands using **ZiRA** (the Dutch Hospital Reference Architecture) as a reusable Industry Architecture reference — illustrating time and design savings from reusing an Industry Architecture.

7.3.3 The Solutions Continuum (pp. 147–149)

The Solutions Continuum focuses specifically on artifacts related to *solutions*, categorizing components/artifacts stored in the Reference Library or Solutions Landscape with increasing detail and specialization. It provides a consistent way to describe/understand the implementation of assets defined by the Architecture Continuum. At each level it is a "population" of the architecture with reference building blocks — purchased products or built components — representing a solution to a business need at that level. A populated repository based on the Solutions Continuum can be regarded as a **solutions inventory**, valuable for managing/implementing improvements.

Moving **left to right** (Figure 7-8, p. 147) focuses on **delivering solution value** (Foundation Solutions → Common Systems Solutions → Industry Solutions → Organization-Specific Solutions). Moving **right to left** focuses on **addressing business needs** (Figure 7-9, p. 148, "Two perspectives of the Solution Continuum"). Both perspectives matter for an organization balancing its needs against maximizing use of available resources through leverage.

The **four solution types**:

- **Foundation Solutions (p. 148):** Highly generic concepts, tools, products, services, and solution components — fundamental providers of capabilities. Includes professional services (training, consulting) that maximize investment value quickly, and support services (e.g., help desk) that maximize value through timely updates/upgrades.
- **Common Systems Solutions (pp. 148–149):** Implementations of a Common Systems Architecture comprised of a set of products/services (possibly certified/branded); represent the highest common denominator across industry segments. They represent collections of common requirements/capabilities rather than customer/industry-specific ones; provide operating environments for operational/informational needs (e.g., high-availability transaction processing, scalable data warehousing). Examples: an enterprise management system product, a security system product.
- **Industry Solutions (p. 149):** Implementations of an Industry Architecture — reusable packages of common components/services specific to an industry. Fundamental components come from Common Systems/Foundation Solutions, augmented with industry-specific components

(e.g., a physical database schema, an industry-specific point-of-service device). Industry-specific, ready to be tailored to individual organizational requirements.

- **Organization-Specific Solutions (p. 149):** Embodiment/implementation of the Organization-Specific Architecture providing required business capabilities; because designed for specific business operations, they contain the **highest amount of unique content** to accommodate varying people/processes.

Building Organization-Specific Solutions on top of Industry, Common Systems, and Foundation Solutions is the **primary purpose of connecting the Architecture Continuum to the Solutions Continuum**. (Further Reading: TOGAF Standard 10th Edition, document C220.)

7.4 Catalogs, Diagrams, Maps, and Matrices (pp. 150–154)

Architecture assets/deliverables stored in the repository consist of catalogs, diagrams, maps, and matrices. The chapter establishes a **containment hierarchy** (Figure 7-10, p. 150): an **architecture deliverable** consists of one or more **architecture artifacts** (standardized products for capturing/visualizing architecture-work information); each artifact consists of one or more **building blocks**; a building block typically consists of one or more architecture entities. The Architecture Repository stores the building blocks.

7.4.1 Catalogs (p. 151)

Catalogs are lists of building blocks of a specific entity, type, or related types, used for governance or reference purposes. A catalog addresses a specific entity (e.g., processes, applications, technology components) and supplements it with additional data; critically, **in a catalog, entities are not linked to other entities** (Table 7-1, p. 151, "Example of a Catalog"). A commonly used example is the **Stakeholder Catalog** (cross-referenced to Chapter 12, Section 12.4, Table 12-8, and Chapter 14, Section 14.3.2, Table 14-7), which lists all stakeholders relevant to the architecture effort along with their concerns, classification, and interest.

7.4.2 Diagrams and Maps (pp. 151–153)

Diagrams and maps are representations of architectural content in either graphical format (diagrams) or textual format (maps); they can be used to graphically populate architectural content

or verify the completeness of collected information. A diagram or map *visualizes information already captured in a catalog or matrix*. Table 7-2 (p. 152, "Example of a Map") and Figure 7-11 (p. 152, "Example of a Diagram") show the *same* information in map (textual/tabular) vs. diagram (graphical) form. A well-known diagram type is the **Process Flow Diagram** (visualizes the steps of a process graphically). A well-known map example is the **Business Capability Map / Organization Map** (cross-referenced to Chapter 12, Section 12.4) — the Business Capability Map lists the capabilities an organization needs, potentially supplemented with processes/applications; the Organization Map shows relationships between the primary entities making up the organization, its partners, and stakeholders.

7.4.3 Matrices (pp. 153–154)

A **matrix** is a grid representing relationships between two or more model entities. Relationships typically spread across two or more catalogs are brought together in a matrix via a technique called **cross-mapping**. Table 7-3 (p. 153) demonstrates cross-mapping Entity A against Entity B, showing which specific values have a direct relationship (e.g., Entity A Value A relates to Entity B Value A but not to Values B or C).

The section closes (p. 154) with a design principle: "**less is more**" — limit the information displayed in any single catalog, diagram, map, or matrix; prefer creating *more* views over overloading existing ones, to preserve readability and usability.

7.5 Architecture Deliverables (pp. 154–175)

Architecture deliverables support conversations about architecture issues and provide insight into situations/scenarios — an essential link for communicating architecture. As established in 7.4, deliverables are composed of artifacts, which are composed of building blocks (catalogs, diagrams, maps, matrices). Figure 7-12 (p. 155) exemplifies this with an **Architecture Definition Document** deliverable composed of three artifacts — an Organization Map, a Process Flow Diagram, and an Application Landscape — each built from one or more building blocks.

7.5.1 Architecture Artifacts (pp. 155–163)

As one moves through ADM phases, a number of architecture artifacts may be produced — but the TOGAF Standard does **not require** creating all listed artifacts; it only indicates possibilities. Architects should use only artifacts useful to the organization/work at hand. Once the Enterprise Metamodel has been adapted to the organization's needs, an overview of necessary artifacts is formed from that model. Figure 7-13 (p. 156, downloadable in large format from eawheel.com/books/media/) shows the possible artifacts tied to the Foundation Enterprise Metamodel, created during the **Preliminary and Requirements Management phases and phases A–E** of the ADM (cross-referenced to Chapter 12, Sections 12.3–12.8 and 12.12).

Table 7-4 (pp. 157–163), "Artifacts and their entities created per ADM Phase," is one of the most exam-relevant tables in the chapter. Key contents by phase:

- **Preliminary Phase (p. 157):** Principles Catalog (entity: Principle).
- **Phase A – Architecture Vision (p. 157):** Stakeholder Catalog; Value Chain Diagram (value stream); Solution Concept Diagram (objective, requirement, constraint); Business Model Diagram (actor, role, stakeholder, contract, function, process, business capability, business service, product); Business Capability Map (business capability); Value Stream Map (value stream).
- **Phase B – Business Architecture (pp. 158–159):** Organization/Actor Catalog; Driver/Goal/Objective Catalog; Role Catalog; Business Service/Function Catalog; Location Catalog; Process/Event/Control/Product Catalog; Contract/Measure Catalog; Business Capabilities Catalog; Value Stream Catalog; Value Stream Stages Catalog; Business Glossary Catalog; plus matrices — Business Interaction Matrix, Actor/Role Matrix, Value Stream/Capability Matrix, Strategy/Capability Matrix, Capability/Organization Matrix; plus diagrams — Business Footprint Diagram, Business Service/Information Diagram, Functional Decomposition Diagram, Product Lifecycle Diagram, Goal/Objective/Business Service Diagram, Business Use-Case Diagram, Organization Decomposition Diagram.
- **Phase C – Data Architecture (p. 160):** Process Flow Diagram, Business Event Diagram, Business Capability Map, Value Stream Map, Organization Map, Information Map, Data Entity/Data Component Catalog, Data Entity/Business Function Matrix, Application/Data Matrix, Conceptual Data Diagram, Data Dissemination Diagram, Data Security Diagram, Data Migration Diagram, Data Lifecycle Diagram.
- **Phase C – Application Architecture (pp. 161–162):** Application Portfolio Catalog, Interface Catalog; Application/Organization Matrix, Role/Application Matrix, Application/Function Matrix, Application Interaction Matrix; Application Communication Diagram, Application and User Location Diagram, Application Use-Case Diagram, Enterprise Manageability Diagram, Process/Application Realization Diagram, Software Engineering Diagram, Application Migration Diagram, Software Distribution Diagram.

- **Phase D – Technology Architecture (pp. 162–163):** Technology Standards Catalog, Technology Portfolio Catalog; Application/Technology Matrix; Environments and Locations Diagram, Platform Decomposition Diagram.
- **Phase E – Opportunities and Solutions (p. 163):** Processing Diagram, Networked Computing/Hardware Diagram, Network and Communications Diagram, Project Context Diagram, Benefits Diagram.

The chapter notes that these artifacts are detailed further in Chapter 12.

7.5.2 Building Blocks (pp. 163–175)

Every artifact from 7.5.1 consists of one or more **building blocks** — described as the "backbone of the architecture," essential reusable components that are modular, generic, and adaptable, enabling architects to combine them in various configurations for efficiency, scalability, and flexibility. A sidebar (p. 164) draws the well-known **LEGO® analogy**: LEGO bricks are generic enough to combine into new/larger sets; TOGAF building blocks work the same way and can be "stacked" — e.g., stacking three Solution Building Blocks creates a larger Architecture Building Block (Figure 7-14, p. 165, "Stackable LEGO bricks"; Figure 7-15, p. 166, "Architecture Building Block composed of multiple Solution Building Blocks").

Building blocks can be defined at varying levels of detail depending on the development stage — early on just a name/brief description, later broken into supporting components with detailed specifications. Formal definition (p. 165): *"A building block represents a potentially reusable component that can be combined with other building blocks to deliver architectures and solutions."* Building blocks are packages of functionality meeting business needs, corresponding to entities defined in the Enterprise Metamodel (organization unit, business service, application, data entity, etc.). A building block is a well-defined unit with clear boundaries, designed to be useful and (preferably) reusable, tangible or intangible. Per the TOGAF Enterprise Metamodel (p. 166), building blocks help identify entity types needed for a logical model, specify a change (Architecture Building Blocks), and create associated physical/operational implementations (Solution Building Blocks). Definition/scope of building blocks should not be tightly bound to implementation, so they remain flexible. The level of detail should align with the architecture's objectives — sometimes a single clear diagram (e.g., a Business Capability Map) is more valuable than a lengthy document.

7.5.2.1 Building Blocks in Architecture Design (pp. 167–168): Formal definition: *"An architecture is a set of building blocks depicted in an architectural model and a specification of how those*

building blocks are connected to meet the overall requirements of the business."* Three core design principles:

- Only include building blocks that directly address the business problem at hand.
- Building blocks often interact in complex ways.
- A single building block may support multiple others, or provide partial support to one.

In early development phases, building blocks tend to be more generalized (for smooth integration); as implementation details emerge, they must become more refined/precise. Each building block should adhere to appropriate standards, Enterprise Principles, and organizational guidelines.

7.5.2.2 Building Block Specification Process (pp. 168–169): Building block definition is an evolutionary/iterative process occurring gradually as the ADM is followed, mainly in **phases A, B, C, and D** (Figure 7-16, p. 168, "ADM Phases and steps at which building blocks are evolved and specified"). Sequence:

- **Phase A (Architecture Vision):** A high-level model of candidate building blocks is created to identify proposed architectural change(s), e.g., required organizational capabilities. Example: a Business Capability Map (created in Phase A, updated/further specified in Phase B).
- **Phases B, C, D:** Clarify which building blocks are needed to further interpret the desired change(s); specify entities that flesh out the Business Capability Map from Phase A. If building blocks don't exist, they are created using catalogs, matrices, and diagrams; if they exist, reuse from the Architecture Repository is preferred. During phases A, B, C, obsolete building blocks are also removed to keep the repository clean/up to date.
- **Phase E (Opportunities and Solutions):** Building blocks are created using work packages (initiatives) that fill gaps for missing building blocks.

The major work across these steps is identifying the **Architecture Building Blocks** required to meet business goals/objectives, then iteratively refining the selected set into **Solution Building Blocks**, which can be bought off-the-shelf or custom developed.

7.5.2.3 Types of Building Blocks (p. 170): Two flavors — **Architecture Building Blocks** (the *what and why*: high-level, technology-agnostic capabilities) and **Solution Building Blocks** (the *how*: practical implementation using specific technologies/products). Figure 7-17 (p. 170) shows an Architecture Building Block implemented *and automated* by a Solution Building Block — a special case: normally Solution Building Blocks only *implement* Architecture Building Blocks, but when the Architecture Building Block is a capability entity and the Solution Building Block is from

the data/application/technology domain, the Solution Building Block can also *automate* the Architecture Building Block.

7.5.2.4 Architecture Building Blocks (pp. 171–173): Created typically during the initial ADM iteration; capture architecture requirements (business, data, application, technology) and guide later Solution Building Block development. They are high-level conceptual components defining *what* needs to be delivered without specifying technology/implementation — concerned with requirements, capabilities, and strategic objectives, independent of specific technologies; help shape architectural vision and form the blueprint for future solutions. Formal definition (p. 171): *"Architecture Building Blocks are architecture documentation and models from the enterprise's Architecture Repository."* Specifications include fundamental functionality/attributes (semantic/unambiguous attributes, security capabilities, manageability), interoperability, relationships to other building blocks, and mapping to business/organizational entities. Example: an organization requiring an HR Management capability (an Architecture Building Block) supported by multiple Solution Building Blocks (organization unit, business service, application, data entity — see Figure 7-15). The capability itself does not specify a particular product/process/service.

Although not explicitly named as such in the TOGAF Standard, Architecture Building Blocks are typically **stored in the Architecture Repository** in three sections (Figure 7-18, p. 172):

- **Architecture Landscape** — stores various architectural models at different abstraction levels, including Architecture Building Blocks defining architecture elements at different development stages.
- **Reference Library** — contains reusable architecture assets (patterns, principles, standards) and Architecture Building Blocks referenceable when developing architectures.
- **Standards Library** — Architecture Building Blocks stored here when aligned with industry standards.

7.5.2.5 Solution Building Blocks (pp. 173–175): Much more detailed than Architecture Building Blocks; describe the entities making up the parent Architecture Building Block. They define *what* products and components will implement the required functionality and determine the actual implementation of (parts of) the architecture. Crucially, Solution Building Blocks are **product- or vendor-aware**, unlike Architecture Building Blocks. Formal definition (p. 173): *"Solution Building Blocks represent actual implementation choices to realize a required capability."* Example: the "Candidate Evaluation Service" (Figure 7-15) is a Solution Building Block. Solution Building Blocks may be procured or developed. Specifications include specific functionality/attributes mapped to IT topology and operational policies (security, manageability, localizability, scalability, performance, configurability), plus relationships to Architecture Building Blocks and other Solution Building

Blocks. Continuing the HR example: the Architecture Building Block (HR Management capability) doesn't specify which application supports it; the Solution Building Block defines the actual application (logical/physical application component) automating the capability. Solution Building Blocks are stored in the **Solutions Landscape** section of the Architecture Repository (Figure 7-19, p. 175), shaped as catalogs, diagrams/maps, or matrices at different abstraction levels. (Further Reading: TOGAF Standard 10th Edition document C220, and TOGAF Series Guide "An Approach to Selecting Building Blocks," document G248.)

7.6 Views and Viewpoints (pp. 176–179)

To communicate architecture correctly, architects must understand stakeholder concerns — this requires distinguishing two similar-but-fundamentally-different concepts:

- **View (p. 176):** A representation of a system or architecture from the perspective of a specific stakeholder — a visualization/presentation addressing particular concerns/interests of one or more stakeholders. Purpose: communicate architectural decisions or system properties meaningfully. Example: a security view describes security measures; a process view shows business process flow.
- **Viewpoint (p. 176):** Defines the perspective and rules for creating a view — essentially a template/set of guidelines for constructing views to address specific concerns; outlines how to structure views and which stakeholders they target. Ensures views are created consistently and address relevant stakeholder concerns. Example: a performance viewpoint specifies how to create views dealing with response times/throughput.

7.6.1 Relationship Between Views and Viewpoints (pp. 177–179)

A viewpoint is like a **blueprint** for constructing views — multiple views can be created from a single viewpoint, but each view is tailored to specific stakeholder concerns. In summary: **a view is what you present/show to stakeholders; a viewpoint is the set of rules/guidelines dictating how you create that view.** Views and viewpoints belong to the **Reference Library** of the Architecture Repository.

Two illustrative examples clarify this:

- **The village/binoculars analogy (Figure 7-20, p. 177):** A village in a valley is observed from two mountaintops via binoculars. The people are stakeholders; the binoculars are viewpoints; the village itself is the view. Each observer focuses on a different feature (church tower vs. green park) — same view, different viewpoints, due to position/interest.
- **The EHR real-world example (Figure 7-21, p. 178):** A physician and a patient both view the same Electronic Health Record (EHR) system. The physician's viewpoint focuses on managing patient information, supporting real-time decision-making, and streamlining administrative tasks. The patient's viewpoint focuses on accessing their own medical information, communicating with providers, and managing health conveniently. Because different stakeholders view the same system differently (due to constraints, requirements, or concerns), it is important to create **multiple viewpoints for the same view**. (Further Reading: TOGAF Standard 10th Edition, document C220.)

7.7 Using the Architecture Repository (pp. 179–183)

Because the Architecture Repository is intangible, it can be difficult to translate into everyday practice — but it can be made tangible via an **architecture tool**. Most tools document/store catalogs, diagrams, maps, matrices, and can store Architecture and Solution Building Blocks. Small organizations may be tempted to use word processors, presentation software, or spreadsheets, but these lack a unified modeling language and force manual tracking of every artifact independently — unsustainable as the organization/architecture grows. Tools offering database-like construction come closer to what's needed; the key requirement is the ability to **relate entities** within the architecture, enabling fast, accurate changes without losing context or redoing work per artifact.

7.7.1 Deciding on a Tool (pp. 180–183)

Even though the repository is conceptually intangible, many organizations use a tool as if it **were** the repository. Vendors offer varying feature sets approximating repository requirements; using an architecture tool is described as "essential." Tool choice depends on factors like **budget** — larger budgets allow market-leading products with fuller functionality; smaller budgets mean less functionality. **Gartner** publishes an annual market-leader report for architecture tools that can inform selection.

Table 7-5 (p. 181), "Architecture tool functionality considerations," lists key evaluation areas: support for a modeling language (e.g., **ArchiMate**, in its entirety); metamodel customization (user-

defined properties on elements/relationships; BPMN support; adjustable metamodel); tool flexibility and stability (colored views; adaptability to personal/organizational needs; concept-changing without losing relationships; performance on large/bulky models); import/export functionality (availability of import/export; making information available to third parties; ability to publish large PDFs as posters/information-exchange artifacts); and support (quality of help desk and user community).

Publishing artifacts to a website (via the tool) helps **increase architecture maturity** — described as "an easy step to higher maturity." Ultimately, tool-selection factors depend heavily on situation; there is no standard, universal requirements set. At minimum, a selected tool should provide a home for artifacts related to the organization's **Architecture and Solutions Landscapes, Reference and Standards Libraries, and Architecture Requirements Repository** (Figure 7-22, p. 182, "Minimum Architecture Repository components that must be represented in an architecture tool"). A repository tool should support only what is minimally needed for EA management — a sidebar quote (p. 183): *"A well-managed repository is characterized by ruthlessly minimizing information collected and maintained."* In practice, most tools fall short of a perfect fit — e.g., **nine out of ten architecture tools do not distinguish between the Architecture Landscape and the Solutions Landscape** (though a good tool allows this distinction via folder structure or similar). The repository's structure should be designed around the organization's artifacts and needs — as simple as a hierarchical folder/subfolder structure — but it must be intuitive and easy to navigate.

7.8 Summary (pp. 183–187)

The chapter's own summary reiterates: the Architecture Repository is a collection of architectures and solutions created during architecture work; a continuum is the lens used to visualize its state. The TOGAF Standard recognizes **three continua**: Enterprise Continuum, Architecture Continuum, Solutions Continuum. The Enterprise Continuum is the umbrella for the other two. The Architecture Continuum has four architecture types (Foundation, Common Systems, Industry, Organization-Specific), increasing in detail left to right. The Solutions Continuum has four solution types (Foundation, Common Systems, Industry, Organization-Specific Solutions), also increasing in detail left to right. Catalogs (lists of building blocks of a specific entity/type, entities not linked to each other), diagrams and maps (graphical/textual representations visualizing catalog/matrix content), and matrices (grids of relationships via cross-mapping) are used to capture repository content. Building blocks are the reusable backbone of architecture, come in two types (Architecture Building Blocks = what/why, technology-agnostic; Solution Building Blocks = how, product/vendor-aware), and are defined iteratively mainly across ADM phases A–D. Views and viewpoints communicate architecture to stakeholders (view = what's shown; viewpoint = the rules for creating it) and live in the Reference Library. Finally, although intangible, the repository is

typically made tangible through an architecture tool, chosen based on situational factors with no universal requirement set.

Key Terms & Definitions

- **Architecture Repository** (p. 128) — An intangible concept (not a tool) in which all aspects of the architecture are collected and stored; should contain only reusable building blocks, an organization-specific metamodel, and architecture governance information.
- **Architecture Metamodel** (p. 129) — Defines the structure for how architecture artifacts are organized and how different types of information (business, data, application, technology, security) relate to each other.
- **Architecture Landscape** (p. 130) — A view of the organization's current, planned, and aspirational architecture, including different levels (strategic, segment, capability) and timelines (past, present, future).
- **Strategic Architectures** (p. 130) — Long-term summary views of the entire organization, providing an organizing framework for operational/change activities and enabling executive-level direction setting.
- **Segment Architectures** (p. 130) — More detailed operating models for areas within an organization, used at program/portfolio level; also called Domain Architectures.
- **Capability Architectures** (p. 130) — Detailed views of how the organization supports a particular unit of capability (current, target, increments), enabling grouping into managed portfolios/programs.
- **Solutions Landscape** (p. 131) — Contains models/documents related to individual Solution Architectures; more detailed than the Architecture Landscape, using Solution Building Blocks.
- **Architecture Requirements Repository** (p. 132) — Captures, manages, and tracks architecture requirements, including both business and technical requirements.
- **Strategic Architecture Requirements** (p. 132) — Forward-looking, executive-level requirements outlining adjustments needed for strategic direction.
- **Segment Architecture Requirements** (p. 133) — Content-focused, portfolio/project-level requirements translating strategy into actionable steps.
- **Capability Architecture Requirements** (p. 133) — Requirements tied to specific capabilities/groups of capabilities for individual initiatives, projects, or work packages.
- **Reference Library** (p. 134) — Contains best practices, guidelines, templates, and patterns reusable across architectural projects; includes the Viewpoint Library.
- **Standards Library** (p. 134) — A collection of standards and policies guiding development/governance of the architecture (IT standards, technology protocols, business process standards, industry guidelines).

- **Governance Repository** (p. 135) — A repository of decisions, rules, governance guidelines, and principles applied to architecture projects, including approvals, waivers, and compliance assessments.
- **Architecture Capability** (p. 135) — The skills, processes, organizational structures, and roles that support the development and management of Enterprise Architecture.
- **Continuum** (p. 136) — A conceptual framework enabling a modular, adaptable approach to shaping an organization's architecture, allowing scaling/customization of development processes.
- **Enterprise Continuum** (p. 138) — Umbrella term covering the Architecture Continuum and Solutions Continuum, bridging conceptual architectures and practical solutions.
- **Architecture Continuum** (p. 138) — Guides creation/evolution of architectures via options from generalized/high-level to specific/concrete implementations.
- **Solutions Continuum** (p. 138) — Works in parallel with the Architecture Continuum, focused on delivering actual real-world solutions/products aligned with the architecture.
- **Foundation Architectures** (p. 142) — The most generalized/abstract architecture type; generic components, interrelationships, principles, and guidelines forming a foundation for more specific architectures.
- **Common Systems Architectures** (p. 143) — Build on Foundation Architectures but remain generic across industries; complete for a particular problem domain but incomplete for overall system functionality.
- **Industry Architectures** (p. 143) — Tailored to a specific industry's unique processes, requirements, and regulations.
- **Organization-Specific Architectures** (p. 144) — The most tailored/customized architecture type, designed for a particular organization's unique needs.
- **Foundation Solutions** (p. 148) — Highly generic concepts, tools, products, services, and solution components; fundamental providers of capabilities.
- **Common Systems Solutions** (p. 148) — Implementations of a Common Systems Architecture comprising products/services; represent the highest common denominator across industry segments.
- **Industry Solutions** (p. 149) — Implementations of an Industry Architecture providing reusable packages of common components/services specific to an industry.
- **Organization-Specific Solutions** (p. 149) — Implementations of the Organization-Specific Architecture providing required business capabilities, with the highest amount of unique content.
- **Architecture Deliverable / Architecture Artifact** (p. 150) — A deliverable is a product created during architecture work, consisting of one or more artifacts; an artifact is a standardized architecture product for capturing/visualizing information related to the architecture work.
- **Building Block** (p. 150; formal def. p. 165) — "A potentially reusable component that can be combined with other building blocks to deliver architectures and solutions."

- **Catalog** (p. 151) — A list of building blocks of a specific entity, type, or related types, used for governance/reference purposes; entities are not linked to other entities.
- **Diagram / Map** (p. 151) — Representations of architectural content in graphical format (diagram) or textual format (map); visualize information already captured in a catalog or matrix.
- **Cross-mapping** (p. 153) — The method of bringing entities together from two or more catalogs into a matrix.
- **Matrix** (p. 153) — A grid representing the relationships between two or more model entities.
- **Architecture** (p. 167) — "A set of building blocks depicted in an architectural model and a specification of how those building blocks are connected to meet the overall requirements of the business."
- **Architecture Building Block (ABB)** (p. 171) — "Architecture documentation and models from the enterprise's Architecture Repository"; high-level, technology-agnostic conceptual components defining what needs to be delivered.
- **Solution Building Block (SBB)** (p. 173) — "Actual implementation choices to realize a required capability"; product- or vendor-aware, detailed implementation components.
- **View** (p. 176) — A representation of a system or architecture from the perspective of a specific stakeholder.
- **Viewpoint** (p. 176) — Defines the perspective and rules for creating a view; a template/set of guidelines for constructing views addressing specific concerns.

Figures & Tables Referenced

- **Figure 7-1** (p. 129) — "The Architecture Repository." The master diagram of the whole repository, showing all key components from top (Architecture Metamodel) to bottom; the backbone reference for the entire chapter.
- **Figure 7-2** (p. 131) — "Excerpt of the Architecture Repository, showing the three levels of architecture granularity." Illustrates Strategic, Segment, and Capability Architectures within the Architecture Landscape.
- **Figure 7-3** (p. 137) — "The Enterprise Continuum." Introduces the overall continuum concept spanning the three continua.
- **Figure 7-4** (p. 139) — "Excerpt of the Architecture Repository, showing the views of the Architecture and Solutions Continua." Shows where in the repository these two continua's perspectives concentrate.
- **Figure 7-5** (p. 140) — "The four classification models of the Enterprise Continuum." Visualizes the scale from most general (foundation) to most specific (organization-specific).

- **Figure 7-6** (p. 142) — "The four architecture types of the Architecture Continuum." Depicts Foundation, Common Systems, Industry, and Organization-Specific Architectures.
- **Figure 7-7** (p. 145) — "Four steps for applying the four architecture types of the Architecture Continuum." A sequential/numbered application of the four types — likely tested as an ordered list.
- **Figure 7-8** (p. 147) — "The four solution types of the Solutions Continuum." Parallels Figure 7-6 for solutions.
- **Figure 7-9** (p. 148) — "Two perspectives of the Solution Continuum." Shows the left-to-right (solution value) vs. right-to-left (business needs) directions of reading the continuum.
- **Figure 7-10** (p. 150) — "Relationship between architecture deliverables, artifacts, and building blocks." Establishes the containment hierarchy central to Sections 7.4–7.5.
- **Figure 7-11** (p. 152) — "Example of a Diagram." Paired with Table 7-2 to contrast diagram vs. map representation of identical information.
- **Figure 7-12** (p. 155) — "Example of an architecture deliverable and its contents." Shows the Architecture Definition Document composed of three artifacts.
- **Figure 7-13** (p. 156) — "Artifacts associated with the Foundation Enterprise Metamodel." Overview map of possible artifacts tied to the metamodel (downloadable at eawheel.com/books/media/).
- **Figure 7-14** (p. 165) — "Stackable LEGO bricks." Analogy figure showing building-block reusability/composability.
- **Figure 7-15** (p. 166) — "Architecture Building Block composed of multiple Solution Building Blocks." Concrete illustration of the ABB/SBB relationship (used again for the HR Management example).
- **Figure 7-16** (p. 168) — "ADM Phases and steps at which building blocks are evolved and specified." Key process figure spanning phases A–E.
- **Figure 7-17** (p. 170) — "Architecture Building Block capability implemented and automated through a Solution Building Block." Shows the special implement-vs-automate distinction.
- **Figure 7-18** (p. 172) — "Excerpt of the Architecture Repository, showing Architecture Building Blocks storage locations." Identifies the three storage sections for ABBs.
- **Figure 7-19** (p. 175) — "Excerpt of the Architecture Repository, showing the Solution Building Blocks storage location." Identifies the Solutions Landscape as the SBB storage location.
- **Figure 7-20** (p. 177) — "Two viewpoints and one view." The village/binoculars analogy diagram.
- **Figure 7-21** (p. 178) — "Two viewpoints and one view in a real-world scenario." The physician/patient EHR example.
- **Figure 7-22** (p. 182) — "Minimum Architecture Repository components that must be represented in an architecture tool." Lists the five minimum components a tool must support.
- **Table 7-1** (p. 151) — "Example of a Catalog." Shows entity/value columns with no cross-entity linkage.

- **Table 7-2** (p. 152) — "Example of a Map." Textual representation of actor/process/business-service relationships, contrasted with Figure 7-11's diagram form.
- **Table 7-3** (p. 153) — "Example of a Matrix." Demonstrates cross-mapping between Entity A and Entity B values.
- **Table 7-4** (pp. 157–163) — "Artifacts and their entities created per ADM Phase." The chapter's largest reference table, enumerating every recommended artifact and its associated metamodel entities per ADM phase (Preliminary through Phase E) — high-value for exam recall.
- **Table 7-5** (p. 181) — "Architecture tool functionality considerations." Lists evaluation criteria (modeling language support, metamodel customization, flexibility/stability, import/export, support/community) for choosing an architecture tool.

Key Takeaways / Exam Focus Points

- The Architecture Repository is explicitly an **intangible concept**, not a tool — a well-designed repository holds only reusable building blocks, an organization-specific metamodel, and governance information, not diagrams of specific solutions (p. 128).
- The repository's key components, read top to bottom in Figure 7-1: Architecture Metamodel, Architecture Landscape (with Solutions Landscape and Architecture Requirements Repository), Reference Library, Standards Library, Governance Repository, and Architecture Capability (pp. 129–136).
- The **Architecture Landscape** has three granularity levels: Strategic, Segment (= Domain), and Capability Architectures — memorize both the names and their scope (long-term/org-wide vs. program/portfolio vs. capability unit) (p. 130).
- The **Architecture Requirements Repository** mirrors the same three-level split for requirements: Strategic, Segment, Capability Architecture Requirements (pp. 132–133).
- Two "libraries" sit beside the Landscape: the **Reference Library** (best practices/templates/patterns + Viewpoint Library) and the **Standards Library** (IT/technology/business/industry standards) — both fed by *external* reference models/standards, e.g., BIZBOK Guide and the TOGAF Standard itself (pp. 134–135).
- The **Architecture Board** provides visibility/direction over both the Governance Repository and the Architecture Capability, and also serves as an escalation mechanism (p. 136).
- There are exactly **three continua**: Enterprise Continuum (umbrella), Architecture Continuum, and Solutions Continuum — reading left to right gives an increasingly detailed perspective of repository contents (p. 137).
- The Enterprise Continuum shows evolution "from generic to specific, from abstract to concrete, and from logical to physical" — a phrase likely to appear verbatim on a quiz (p. 140).

- The **Architecture Continuum's four types**, in order of increasing specificity: Foundation Architectures → Common Systems Architectures → Industry Architectures → Organization-Specific Architectures (pp. 142–144); applied in that same order per Figure 7-7's four numbered steps (p. 145).
- The **Solutions Continuum's four types** parallel the Architecture Continuum exactly: Foundation Solutions → Common Systems Solutions → Industry Solutions → Organization-Specific Solutions (pp. 148–149). Left-to-right = delivering solution value; right-to-left = addressing business needs (pp. 147–148).
- Containment hierarchy: **Architecture Deliverable** → **(one or more) Architecture Artifacts** → **(one or more) Building Blocks** → **(one or more) architecture entities** (p. 150).
- **Catalogs** = lists, no entity linkage. **Diagrams/Maps** = visualize catalog/matrix data graphically (diagram) or textually (map) — same content, different format. **Matrices** = grids showing relationships via **cross-mapping** (pp. 151–153).
- Design guidance: "less is more" — prefer creating additional views over overloading one view with too much information (p. 154).
- **Table 7-4** maps specific artifacts to each ADM phase (Preliminary, A, B, C-Data, C-Application, D, E) — know at least a few signature artifacts per phase, e.g., Principles Catalog (Preliminary), Stakeholder Catalog and Business Capability Map (Phase A), Business Interaction Matrix (Phase B), Data Entity/Data Component Catalog (Phase C-Data), Application Portfolio Catalog (Phase C-Application), Technology Standards Catalog (Phase D), Benefits Diagram (Phase E) (pp. 157–163).
- Building blocks are defined iteratively mainly across **ADM phases A, B, C, and D**; Phase E creates building blocks via work packages/initiatives to fill gaps (pp. 168–169).
- **Two types of building blocks**: Architecture Building Blocks (ABB = what/why, technology-agnostic, high-level) vs. Solution Building Blocks (SBB = how, product/vendor-aware, detailed implementation) — this is one of the most testable distinctions in the chapter (p. 170).
- Normally SBBs only **implement** ABBs, but when the ABB is a capability entity and the SBB is from data/application/technology domains, the SBB can also **automate** the ABB (p. 170).
- ABBs are stored in three repository sections: Architecture Landscape, Reference Library, Standards Library (p. 172). SBBs are stored in one section: the Solutions Landscape (p. 175).
- **View** = what is shown/presented to stakeholders. **Viewpoint** = the rules/template for constructing a view. A single viewpoint can produce multiple views; each view is tailored to specific stakeholder concerns. Both belong to the Reference Library (pp. 176–177).
- The village/binoculars analogy and the physician/patient EHR example both illustrate: same view, different viewpoints depending on stakeholder position/interest/concerns (pp. 177–178).
- Even though intangible, the repository is made tangible via an **architecture tool**; word processors/spreadsheets/presentation software are inadequate at scale because they lack unified modeling languages and force manual artifact tracking (pp. 179–180).

- **Table 7-5** tool-selection criteria: modeling language support (e.g., ArchiMate, BPMN), metamodel customization, tool flexibility/stability, import/export functionality, and support/community (p. 181).
- At minimum, a tool must represent: Architecture Landscape, Solutions Landscape, Reference Library, Standards Library, and Architecture Requirements Repository (Figure 7-22) (p. 182).
- Fact likely to be quizzed: **nine out of ten architecture tools do not distinguish between the Architecture Landscape and the Solutions Landscape** (p. 183).
- Chapter mnemonic: **E-A-S** continua (Enterprise, Architecture, Solutions) and **F-C-I-O** architecture/solution types (Foundation, Common Systems, Industry, Organization-Specific) — both four-step progressions from generic to specific (pp. 137, 142–144, 148–149).

Self-Check Questions

Q: According to the TOGAF Standard, what three things should a well-designed Architecture Repository contain, and what should it explicitly NOT contain?

A: It should contain reusable building blocks, an organization-specific metamodel, and additional architecture governance information. It should not contain diagrams of specific solutions. (p. 128)

Q: What are the three levels of granularity into which the Architecture Landscape is divided, and which one is also known as "Domain Architecture"?

A: Strategic Architectures, Segment Architectures, and Capability Architectures. Segment Architectures are also called Domain Architectures. (p. 130)

Q: What is the key difference between the Architecture Requirements Repository's Segment Architecture Requirements and Strategic Architecture Requirements?

A: Strategic Architecture Requirements are forward-looking, executive-level, and outline the organization's overarching direction; Segment Architecture Requirements are more precise, content-focused, portfolio/project-level requirements that translate strategic goals into concrete, actionable steps ("tactical enablers"). (pp. 132–133)

Q: Name the three continua defined by the TOGAF Standard and state which one acts as the umbrella term.

A: Enterprise Continuum, Architecture Continuum, and Solutions Continuum; the Enterprise Continuum is the umbrella term covering the other two. (p. 138)

Q: List, in order from most generic to most specific, the four architecture types of the Architecture Continuum.

A: Foundation Architectures, Common Systems Architectures, Industry Architectures, Organization-Specific Architectures. (pp. 142–144)

Q: What distinguishes a catalog from a matrix in the way it represents entities?

A: A catalog is a list addressing a specific entity/type where entities are NOT linked to other entities; a matrix is a grid representing relationships between two or more model entities, brought together via cross-mapping. (pp. 151, 153)

Q: What is the containment hierarchy connecting architecture deliverables, artifacts, and building blocks?

A: An architecture deliverable consists of one or more architecture artifacts; each artifact consists of one or more building blocks; a building block typically consists of one or more architecture entities. (p. 150)

Q: What is the fundamental distinction between an Architecture Building Block and a Solution Building Block?

A: An Architecture Building Block represents the "what and why" — high-level, technology-agnostic capabilities; a Solution Building Block represents the "how" — practical implementation using specific technologies/products, and is product- or vendor-aware. (p. 170)

Q: In which sections of the Architecture Repository are Architecture Building Blocks typically stored, versus where are Solution Building Blocks stored?

A: Architecture Building Blocks: Architecture Landscape, Reference Library, and Standards Library. Solution Building Blocks: the Solutions Landscape. (pp. 172, 175)

Q: During which ADM phases is the building block specification process mainly carried out, and what happens during Phase E specifically?

A: Mainly during phases A, B, C, and D. During Phase E (Opportunities and Solutions), building blocks are created using work packages (initiatives) that fill gaps for missing building blocks. (pp. 168–169)

Q: Define "view" and "viewpoint" and explain their relationship.

A: A view is a representation of a system/architecture from the perspective of a specific stakeholder (what is presented). A viewpoint defines the perspective and rules for creating a view (a template/guideline). A viewpoint is like a blueprint — multiple views can be created from a single viewpoint, each tailored to specific stakeholder concerns. (pp. 176–177)

Q: According to the chapter, what proportion of architecture tools fail to distinguish between the Architecture Landscape and the Solutions Landscape, and what commonly compensates for this?

A: Nine out of ten architecture tools do not distinguish between them; a good tool allows the user to make this distinction based on a folder structure or something similar. (p. 183)

Q: What are the four solution types of the Solutions Continuum, and what does moving left-to-right versus right-to-left represent?

A: Foundation Solutions, Common Systems Solutions, Industry Solutions, and Organization-Specific Solutions. Left-to-right focuses on delivering solution value; right-to-left focuses on addressing business needs. (pp. 147–149)